

版本控制 · 远程托管 · 分支协作

Git 与 GitHub

使用指南

从第一次提交，到一次完整协作。以纯文本项目为例，学习版本控制、本地命令、GitHub 托管、分支与 Pull Request，以及账号和代码安全。

学习目标 独立完成本地提交、远程推送与拉取、分支合并和 Pull Request，并在提交前检查敏感信息。

内容导航

内容	学习任务
环境与认证	安装 Git，配置提交身份，完成 SSH 认证。
本地版本控制	初始化、暂存、提交、比较差异与查看历史。
分支与协作	创建分支、合并修改、提交并合并 Pull Request。
GitHub 托管	创建仓库、推送、克隆与拉取远程更新。
安全与排错	忽略敏感信息，保护账号，识别常见错误。
命令与资料	查询常用命令及其作用范围，查阅官方文档。

使用约定

示例使用 Windows Git Bash、macOS Terminal 或 Linux Bash。Git 命令可用于其他终端，但 printf、cat 和 <<'EOF' 写文件语法不能原样粘贴到 Windows CMD 或 PowerShell。

YOUR_USERNAME 表示仓库所属账号；YOUR_NOREPLY_EMAIL 表示完整提交邮箱。运行前替换为自己的信息。Git Demo 是示例作者名，可改为自己的显示名。命令块不包含终端提示符。

练习仅使用新建目录、个人练习仓库与无效占位数据；不要公开真实课程答案或提交真实凭据。

参考：[A][B]；Git 与 GitHub 官方文档索引见本指南末尾。

安装 Git 与选择终端

系统	安装方式	终端环境
Windows	从 Git 官网安装 Git for Windows。	打开 Git Bash；不要把本指南整段写文件语法放进 CMD。
macOS	先运行 <code>git --version</code> ；缺失时安装命令行工具，或按 Git 官网方式安装。	Terminal 的 <code>zsh</code> / <code>bash</code> 均可运行示例命令。
Ubuntu / Debian	通过系统包管理器安装 Git 与 OpenSSH 客户端。	Terminal / <code>bash</code> 。

按操作系统选择安装命令

```
# macOS: only if command-line tools are missing
xcode-select --install

# Ubuntu / Debian only
sudo apt update
sudo apt install git openssh-client
```

所有系统的检查命令

```
git --version
ssh -V
pwd
```

本例需要 Git 2.28 或更新版本，以支持 `git init -b main`；建议使用操作系统支持的更新版本。请看 `git --version` 的输出，不要把“没有报错”当作已经完成 GitHub 认证。

Windows 路径示例：F:\test 在 Git Bash 中写作 `/f/test`；含空格的目录需要双引号。主线统一在用户主目录下，避免盘符差异。

命令块按步骤执行。上一条失败就停下来排查，尤其是 `cd / mkdir`；不要在错误目录继续 `git init`、写文件或 `commit`。

参考：[8][20][22]；安装入口以官方页面为准。

02 / SSH 密钥

SSH 认证①：生成或复用密钥

示例采用 SSH 进行认证。配置 user.name / user.email 不会自动获得仓库权限。HTTPS + GCM 是另一种认证方式，见“认证与网络排错”。

先检查已有密钥

```
ls -al ~/.ssh
```

先查看已有文件名。若目录不存在，可以生成密钥；已有可用的 id_ed25519 与 id_ed25519.pub 时不要覆盖，可直接复用并验证账号。不同账号应使用独立命名的密钥，或配置对应的 HTTPS 认证。

没有现成密钥时，再生成

```
ssh-keygen -t ed25519 -C "git-class"
```

按提示选择保存路径；只有默认路径没有同名密钥时才接受默认路径。随后设置并确认密钥口令。-C 后的 git-class 只是密钥备注标签，不是 GitHub 登录信息。

文件	可否对外提供	用途
id_ed25519.pub	可以添加到 GitHub 账号。	公钥：用于让服务端验证你的身份。
id_ed25519	不可以展示、提交或发送。	私钥：留在本机，由 SSH 客户端使用。

若保存了自定义文件名，后面的 ssh-add 与 cat 路径必须同步替换。出现“是否覆盖已有文件”时不要输入 yes。

参考：[9][13]。

SSH 认证 ②：添加与验证

加载私钥，只复制公钥

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
cat ~/.ssh/id_ed25519.pub
```

在 GitHub 中打开头像 → Settings → SSH and GPG keys → New SSH key。Title 写明设备；Key type 选择 Authentication Key；粘贴 .pub 的整行内容并添加。菜单名称可能调整，以实际页面为准。

ssh-add 使用本地私钥文件，不显示文件内容；cat 命令只读取 .pub 公钥。macOS 需要长期保存口令时，可按官方说明使用系统 ssh-add 的 --apple-use-keychain 选项。

测试连接

```
ssh -T git@github.com
```

首次连接可能询问是否信任主机。先用官方“GitHub SSH key fingerprints”页面核对主机指纹，完全匹配再确认。不匹配就停止；不要关闭主机校验来消除提示。

看到“**successfully authenticated**”且问候中的用户名是你的账号，就说明 SSH 身份认证成功。后面的“**does not provide shell access**”是正常说明；该测试命令的退出码可以是 1。

认证成功不代表拥有某个仓库的写权限。GitHub 仓库 owner、仓库名及你的账号权限仍需正确。换终端后如再次要求口令，可重新启动 agent 并 ssh-add。

参考：[9][10][11]；主机指纹请实时核对官方页面，不依赖讲义中的固定值。

04 / 本地仓库

创建仓库与配置提交作者

项目名统一为 git-class-demo。以下本地步骤不需要联网；示例只含文本文件，无需安装项目依赖。

先确认主目录中不存在 git-class-demo。若同名目录已存在，停止本步骤并换新名字；不要在原目录中覆盖 README 或 notes。

```
cd ~
mkdir git-class-demo
cd git-class-demo
git init -b main
git config user.name "Git Demo"
git config user.email "YOUR_NOREPLY_EMAIL"
git config --get user.name
git config --get user.email
```

把 YOUR_NOREPLY_EMAIL 替换为 GitHub Settings → Emails 中提供给你的完整 noreply 地址，也可以使用已关联到账号的邮箱。不要自行拼接 noreply；邮箱会写入未来的提交记录。

命令或词	含义
git init -b main	把当前目录初始化为仓库，明确初始分支叫 main。
git config user.name	记录提交的作者显示名，不是 GitHub 登录账号。
git config user.email	记录提交邮箱；不承担认证功能。
不加 --global	只配置当前仓库，避免影响其他项目。
.git	保存 Git 管理数据；不手工编辑或删除。

检查结果

git branch --show-current 应显示 main；两条 config --get 输出配置的名字和邮箱。不带 --global 的配置只作用于当前仓库。

参考：[8][12][20]。

创建示例文件与忽略规则

当前目录为 ~/git-class-demo。下面的 > 会覆盖同名文件，只在新练习目录中执行。API_KEY 均为无效占位数据，不具有真实权限。

```
printf '# Git Class Demo\n\nA small repository for class.\n' > README.md
printf 'Today I learned Git.\n' > notes.txt
cat > .gitignore <<'EOF'
.env
.env.*
!.env.example
*.log
.DS_Store
EOF
printf 'API_KEY=REPLACE_WITH_YOUR_OWN_VALUE\n' > .env.example
printf 'API_KEY=DEMO_ONLY_NOT_A_REAL_KEY\n' > .env
git status --short
```

文件	本次是否提交
README.md / notes.txt	提交；分别是项目说明与学习笔记。
.gitignore	提交；团队共享忽略规则。
.env.example	提交；只保留字段名与占位值。
.env	不提交；本例只放假值，用于演示忽略规则。

看到 README.md、notes.txt、.gitignore、.env.example 前面的 ??，说明尚未跟踪。正常情况下 .env 不出现在列表中。

<<'EOF' 表示把后续多行按原样写入文件；结束的 EOF 必须单独一行。> 覆盖文件，>> 追加文件。本指南使用英文半角引号。

参考：[1][4][17]。

完成第一次提交

```
git add README.md notes.txt .gitignore .env.example
git status --short
git diff --cached
git commit -m "Initial commit"
git status
git --no-pager log --oneline
```

观察	预期结果
git status --short	暂存后四个文件标记为 A；.env 不在其中。
git diff --cached	展示将进入本次提交的内容；仔细检查没有凭据。
git commit	出现提交编号、Initial commit 和文件变化摘要。
git status	显示工作区干净；被忽略的 .env 仍可以留在本地。
git log --oneline	出现一条提交记录。提交编号因机器和时间不同而变化。

暂存、提交与远程同步

add 将选中文件当时的内容放入暂存区；commit 将暂存区快照写入本地历史。此时尚未执行 push，GitHub 不会更新。提交说明是提交记录的一部分，commit 本身不是上传。

普通 git diff 比较工作区和暂存区；git diff --cached 比较暂存区和当前提交。刚创建、尚未跟踪的文件通常不显示在普通 diff 中，先用 status 找到它们。

```
git add . 很常见，但会暂存当前目录范围内未被忽略的变更。本次使用明确文件名，方便新手检查边界。
git commit -a 不会自动加入全新的未跟踪文件。
```

进入分页界面时可按 q 退出。git --no-pager log 直接输出日志，不启动分页器。

参考：[A] 第 10 页；[1][17]。

修改文件并再次提交

```
printf 'I can track changes.\n' >> notes.txt
git diff
git add notes.txt
git diff --cached
git commit -m "Update learning notes"
git --no-pager log --oneline
git show HEAD:notes.txt
```

理解两种差异

git diff 显示工作区与暂存区的差异；git diff --cached 显示下一次提交的内容。完成 commit 后，git log 可查看前后两条提交记录。

词	含义
HEAD	在本例常规分支操作中，代表当前检出的提交位置。
HEAD:notes.txt	读取当前提交中的 notes.txt，不是随意打开磁盘文件。
提交编号	标识一个提交；每次提交都有对应的标识。
提交说明	写清楚改了什么；一次提交尽量完成一个小目标。

暂存不是实时同步：add 之后再修改同一个文件，新修改不会自动进入已暂存的快照。需要重新 add，或有意分成两次提交。

此时仍然完全在本地。关闭终端不会抹去提交，但只存在这一台设备中的仓库也不是完整的异地备份方案。

参考：[1][7]。

创建分支并合并修改

```
git switch -c feature/intro
printf 'Hello from a feature branch.\n' > intro.txt
git add intro.txt
git commit -m "Add introduction"
git switch main
ls
git merge --no-ff feature/intro -m "Merge intro"
git --no-pager log --oneline --graph --all
```

创建分支后，在 feature/intro 上提交 intro.txt；切回 main，再用 ls 观察文件暂时不在当前版本中。合并后文件回到 main，日志图能看到分支和合并提交。

命令	含义
git switch -c feature/intro	创建并切换；等价于先 branch 再 switch。
git switch main	只切换到已有分支。
git merge feature/intro	把指定分支整合进“当前所在分支”。
--no-ff -m "Merge intro"	保留合并提交，并使用 -m 指定提交说明。

分支与部署的区别

分支不是复制了一整个项目目录，而是指向提交的轻量引用。main 只是本例主分支的名字，不等于线上正在运行的生产环境。合并代码和部署上线是不同动作。

main 有新的提交，不代表合并必然冲突。Git 会尝试整合；同一位置有不兼容修改等情况才可能需要人处理。切分支前先保持工作区干净。

参考：[A] 第 12 页；[3][7]。

创建空的远程仓库

进行远程操作前，需完成 GitHub 注册、邮箱验证与 SSH 认证。浏览器登录账号应与目标仓库的 Owner 一致。

网页操作	选择
右上角 + → New repository	进入创建页面。
Owner	选择自己的账号，避免误用组织仓库。
Repository name	git-class-demo；已存在时换名，并同步修改本地命令。
Description	Git practice repository。
Visibility	Private。
Add a README file	关闭 / 不勾选。
Add .gitignore	None / 不添加。
Choose a license	None / 不添加。
Create repository	确认创建；进入空仓库的快速设置页面。

选择空仓库的原因

本地已生成首次提交。若网页再次生成 README 等初始提交，本地与远程可能具有互不关联的起点。“先本地创建、再托管”的流程使用空远程仓库。

另一条合法路线是“GitHub 先生成 README → git clone → 在克隆目录里修改”。两条路线二选一，不要把本地独立 init 与网页独立初始化混在一起。

创建页面应确认四项信息：Owner、仓库名、可见性，以及是否初始化文件。按钮名称与位置以实际页面为准。

参考：[B] 第 1–2 页；[8]。

10 / 推送与克隆

推送本地提交并克隆仓库

在 `~/git-class-demo` 中，把 `YOUR_USERNAME` 换成仓库 owner 的真实用户名。SSH 地址中的 `git@` 不要改成你的用户名。

```
git remote add origin git@github.com:YOUR_USERNAME/git-class-demo.git
git remote -v
git push -u origin main
git branch -vv
git rev-parse --short HEAD
```

`origin` 是给远程地址起的常用别名；`main` 是要推送的本地分支；`-u` 设置上游跟踪关系，之后在 `main` 上通常可以直接 `git push`。此次只推 `main` 及它可达的提交，并非上传工作目录中的所有文件或所有本地配置。

检查仓库页面

刷新仓库的 Code 页面，确认当前分支为 `main`；看到 README、notes、intro 和忽略规则等已提交内容；打开提交历史，核对最新提交说明与短哈希。确认 `.env` 不存在于文件列表中。

克隆到新目录

```
cd ~
git clone git@github.com:YOUR_USERNAME/git-class-demo.git git-class-clone
cd git-class-clone
cat notes.txt
cd ~/git-class-demo
```

第二份副本名为 `git-class-clone`，它不能与已有目录冲突。最后一条 `cd` 返回原仓库，后续主线都在原仓库继续。私有仓库 clone 需要认证和读权限。

push 不是“把文件夹完整上传”。未提交的修改、被忽略的 `.env`、GitHub 的 Issue / PR 等平台信息都不能简单理解为随 push 原样搬过去。

参考：[8][21]。

同步 GitHub 网页提交

先在 GitHub 中操作

进入自己的练习仓库，切到 main，打开 notes.txt，点击编辑，在末尾新增一行 Edited on GitHub.。点击 Commit changes，说明写 Update notes from GitHub，选择直接提交到 main 并确认。个人练习仓库可直接提交到 main；团队仓库可能要求走 PR。

回到原来的本地仓库

```
cd ~/git-class-demo
git switch main
git status
git pull --ff-only
cat notes.txt
git --no-pager log --oneline -5
```

观察	解释
pull 前 status 干净	减少把未提交修改带入整合的风险。
pull 显示更新	远程新提交已拉取，并把本地 main 快进到新位置。
cat 出现新增的一行	内容从 GitHub 回到了本地。
log 中出现网页提交	网页编辑也是提交，不只是改网页显示。

pull 的含义是先 fetch 再整合，不是“强制覆盖本地”。本例明确使用 --ff-only：有分叉时停止，不替你选择复杂整合策略。

不能快进时，先查看“目录、分支与提交排错”，判断本地是否存在额外提交。不要直接强推覆盖远程历史。

参考：[2]。

准备 Pull Request

继续使用原仓库，确认刚刚已把 main 同步到远程最新状态。新功能使用新的 feature/checklist 分支，而不是复用刚合并的 feature/intro。

```
git switch -c feature/checklist
printf 'Check status before commit.\nReview diff before push.\n' > checklist.txt
git add checklist.txt
git diff --cached
git commit -m "Add demo checklist"
git push -u origin feature/checklist
```

分支推送与合并的区别

检查清单在 feature/checklist 分支中提交。push 仅将这个分支同步到 GitHub，不会自动改变 main；合入主分支需要创建、审阅并合并 Pull Request。

概念	不要混淆
push	把本地分支相关提交发送到远程。
Pull Request / PR	GitHub 上提出、讨论和审查分支更改的流程。
git pull	把远程提交取回并整合到本地，和 PR 不是同一命令。
base / compare	base 是目标分支；compare 是提供更改的分支。

在自己拥有的仓库中可直接推送功能分支。向没有写权限的他人项目贡献时，通常先 fork，再从自己的副本发起 Pull Request。

参考：[15][21]。

创建、合并并同步 Pull Request

顺序	GitHub 网页操作
1	进入 Pull requests → New pull request；或使用 Compare & pull request 提示。
2	确认 base: main，compare: feature/checklist，方向不能反。
3	检查 Files changed，只应出现预期的 checklist.txt 更改。
4	Create pull request；填写标题与改动目的，不上传凭据或无关文件。
5	满足仓库要求后，选择 Create a merge commit / Merge pull request，确认合并。
6	回 Code 的 main 分支查看 checklist.txt，确认已出现。

个人练习仓库可由自己完成流程。团队协作应满足评审与分支保护规则；仓库只允许 squash / rebase 时，以仓库规则为准，保留本地功能分支即可。

拉回合并结果

```
git switch main
git pull --ff-only
cat checklist.txt
git --no-pager log --oneline --graph --all -8
git branch -d feature/checklist
```

main 同步后可查看 checklist.txt。git branch -d 是安全删除本地已合并分支的检查式命令；它不会删除远程分支。选择“创建合并提交”时，分支祖先关系被保留。

使用 Squash and merge 后，branch -d 可能报告“未合并”。应先保留并检查该分支，不要直接强制删除。

参考：[15][16][3]。

区分三类撤销方式

在 git-class-demo 的干净工作区中练习。下面的临时修改和 scratch.txt 均为一次性练习数据。

① 加错到暂存区：撤销暂存，保留编辑

```
printf 'TEMPORARY CLASSROOM EDIT\n' >> notes.txt
git add notes.txt
git restore --staged notes.txt
git diff
git diff --cached
```

git diff 仍看到临时行，git diff --cached 为空：说明修改没有丢，只是移出了下一次提交。

② 放弃刚才那一行：恢复工作区

下面的 restore 会丢弃 notes.txt 中尚未暂存的修改。确认只有上一段造出的临时行，再执行；真实工作先备份。

```
git restore notes.txt
git status
```

③ 已提交：用新提交反向撤销

```
printf 'A disposable demo file.\n' > scratch.txt
git add scratch.txt
git commit -m "Add scratch file"
git revert --no-edit HEAD
git --no-pager log --oneline -3
```

这里的 HEAD 是刚生成的普通提交，不是 merge commit。revert 新建反向提交，旧记录仍保留；撤销合并提交需要额外判断。

共享分支不应随意改写历史；使用丢弃修改或强制覆盖类命令前，应确认作用范围并备份。

参考：[5][6]。

创建冲突并完成合并

另建一个全新目录 git-conflict-lab；同名已存在就换名。本实验不添加远程，不影响主线 GitHub 仓库。它故意让两个分支修改同一行。

```
cd ~
mkdir git-conflict-lab
cd git-conflict-lab
git init -b main
git config user.name "Git Demo"
git config user.email "demo@example.invalid"
printf 'title=original\n' > title.txt
git add title.txt
git commit -m "Add original title"
git switch -c feature/title
printf 'title=feature\n' > title.txt
git add title.txt
git commit -m "Change title on feature"
git switch main
printf 'title=main\n' > title.txt
git add title.txt
git commit -m "Change title on main"
git merge feature/title
```

最后一条 merge 会报告冲突并暂停合并。文件中出现冲突标记后，需确定最终内容，再暂存并完成提交。

```
cat title.txt
printf 'title=main-and-feature\n' > title.txt
git add title.txt
git commit -m "Resolve title conflict"
git status
```

真正的项目要先理解双方改动并测试，而不是直接保留任意一边。需要退出当前合并时，且合并前工作区干净，可用 `git merge --abort`。解决后是 `add` → `commit`，不是再盲目运行同一次 `merge`。

参考：[3]。

避免提交真实凭据

回到原来的 git-class-demo。该仓库从第一次提交前就准备了 .gitignore；它是减少误提交的工具，不是加密或访问控制系统。

```
cd ~/git-class-demo
git check-ignore -v .env
git ls-files -- .env
git diff --cached
git status --short
```

检查	预期
check-ignore -v .env	输出命中的规则及其来源。
ls-files -- .env	没有输出，说明当前索引里不跟踪 .env。
diff --cached	确认待提交内容，不含真实密钥、Token、密码或隐私。
.env.example	只含字段名和占位符，便于同伴配置。

已经跟踪的文件怎么办

只有 git ls-files -- .env 确实有输出，且已备份需要的文件时，才考虑下面的操作；主线仓库正常情况下不需要执行。它把 .env 从索引移除并保留本地文件，随后提交这一变化，但不会清除旧历史。

```
git rm --cached .env
git add .gitignore
git commit -m "Stop tracking local environment file"
```

如果历史中已经出现真实凭据，即使还没公开，也不能把新增 .gitignore 当作处理完成。按实际暴露范围通知负责人，并优先撤销或轮换凭据。

参考：[4][17]。

删除文件不会清除历史

新建独立目录 git-history-safety-lab，不连接 GitHub。同名目录已存在时应换名。该例不创建忽略规则，仅提交一个无效占位值以观察历史。

```
cd ~
mkdir git-history-safety-lab
cd git-history-safety-lab
git init -b main
git config user.name "Git Demo"
git config user.email "demo@example.invalid"
printf 'API_KEY=DEMO_ONLY_NOT_A_REAL_KEY\n' > .env
git add .env
git commit -m "Add a fake classroom value"
git rm .env
git commit -m "Delete the fake value file"
ls -a
git show HEAD~1:.env
```

当前文件与历史记录

当前目录已经没有 .env，但 git show HEAD~1:.env 仍能显示上一提交里的假值。Git 的目标就是保留历史，因此“删掉后再提交”不等于让旧内容消失。

仅使用 DEMO_ONLY_NOT_A_REAL_KEY 等无效占位值。不要提交真实 API Key、SSH 私钥、Token 或口令，也不要将该练习仓库推送到线上。

真实泄露时的顺序

先在凭据提供方撤销或轮换 → 检查访问记录与影响范围 → 按团队流程协调清理历史和相关副本 → 补上预防措施。改写历史有协作风险，也不能保证删除他人的克隆、截图或所有缓存。

参考：[17]。

保护账号、仓库和课程材料

账号层

为 GitHub 配置双因素认证并妥善保存恢复方式。公钥可添加到账号，私钥和恢复码不可外传。离开共享设备前退出账号，不在公共电脑上长期保留 SSH 私钥或登录会话。

仓库层

本例选择 Private。只邀请确实需要访问的人，给适当的权限。私有仓库仍可能因为误授权、账号被盗或协作者转发而泄露，不能把它当成放置真实密钥的保险箱。

课程层

课程作业不应放到可公开访问的版本库，以免产生抄袭与学术诚信问题。具体提交与共享范围以任课老师要求为准。

代码层

clone 只是把仓库取到本地，不等于其内容安全。对于不熟悉的项目，先检查来源、README、安装脚本和依赖说明，再决定是否在隔离练习环境运行；不要为了跟做教程直接执行来历不明的脚本。

安全原则 提交前看 diff，公开前看权限；凭据不进库，泄露先撤销。

参考：[A] 第 8 页；[17][18]。

认证与网络排错

现象	先检查	处理方向
Permission denied (publickey)	ssh -T 的账号；ssh-add -l；公钥是否在 GitHub。	在当前终端启动 agent、加载正确私钥；核对仓库权限。
Repository not found	git remote -v 中 owner / 仓库名；当前账号读权限。	私有仓库无权限也可能显示不存在；不要只改密码。
SSH 22 端口超时	浏览器能否正常访问 GitHub；校园网络是否允许 SSH。	使用 HTTPS + GCM，或测试官方 SSH 443 端口方式。
HTTPS 提示认证失败	是否错误地输入 GitHub 网页登录密码。	Git HTTPS 不使用网页密码；用 GCM/CLI 浏览器认证或合适的 Token。
主机指纹不匹配	访问主机名与官方指纹。	立即停止核验；不关闭校验，不盲删 known_hosts。

HTTPS 方式：使用 GCM 管理凭据

```
git remote set-url origin https://github.com/YOUR_USERNAME/git-class-demo.git
git push -u origin main
```

由凭据管理器打开浏览器认证，不将 Token 写进 URL、命令历史或截图。已安装 GitHub CLI 时，也可使用 gh auth login，选择 HTTPS 并允许配置 Git 认证。

SSH 443 方式

```
ssh -T -p 443 git@ssh.github.com
git remote set-url origin ssh://git@ssh.github.com:443/YOUR_USERNAME/git-class-demo.git
```

先测试连接并核验主机指纹，成功后再修改地址。这是 443 端口上的 SSH，不是普通 HTTPS；是否可用取决于网络环境。

参考：[10][11][13][14][19][21]。

本地状态与同步排错

现象	原因与处理
not a git repository	先 pwd，再 cd 到实际仓库；不要在错误目录补一个 init 来掩盖问题。
Author identity unknown	在当前仓库配置 user.name 与 user.email，再提交。
src refspec main does not match any	确认已有第一次提交，且当前分支确实是 main；看 log 和 branch。
remote origin already exists	先 git remote -v；只有地址确实错误才用 remote set-url。
non-fast-forward / cannot fast-forward	本地与远程不在简单快进关系；先取回并查看历史，不要强推。
nothing to commit	看 status：可能未保存、未 add、在错误目录，或根本没有新改动。
LF will be replaced by CRLF	通常是换行符策略信息，不等于提交失败；按项目约定设置换行符。
日志停在屏幕里	按 q 退出分页；或使用 git --no-pager log。

共享祖先但有分叉：先观察，再整合

```
git status
git fetch origin
git --no-pager log --oneline --graph --all -12
```

先保持工作区干净。确认是同一项目、同一 main 分支的历史分叉后，可用 git merge origin/main 整合。发生冲突时按“合并冲突”章节处理，完成 commit 后再 push。

网页与本地各自初始化，历史无关

新手优先从已有远程仓库克隆到另一个全新目录，复制需要的项目文件进去再提交；不要复制旧 .git，也别直接使用强推或强行拼接历史。原目录保留作备份。

排查顺序 确认目录 → 查看状态 → 核对分支与远程 → 获取并比较历史。保留原数据，不通过强制覆盖掩盖问题。

参考：[2][3][8][21]。

常用命令与作用范围

任务	命令	改变哪里
新仓库	git init -b main	当前目录的 Git 管理信息。
看状态	git status	只观察。
看未暂存差异	git diff	只观察工作区与暂存区。
暂存	git add 文件名	暂存区。
看待提交内容	git diff --cached	只观察暂存区与当前提交。
提交	git commit -m "说明"	本地历史。
看历史	git log --oneline --graph --all	只观察。
新分支并切换	git switch -c 分支名	本地分支与检出位置。
切分支	git switch main	检出位置与对应工作区。
合并	git merge 分支名	当前分支。
绑定远程	git remote add origin 地址	本地远程配置。
首次推送	git push -u origin main	远程分支；并记录本地上游关系。
取一份仓库	git clone 地址 新目录	新本地仓库。
拉取并快进	git pull --ff-only	取回远程并在可快进时更新当前分支。
撤销暂存	git restore --staged 文件名	暂存区，不丢工作区编辑。
反向撤销普通提交	git revert 提交编号	新建一条反向提交。
检查忽略	git check-ignore -v 文件名	只观察匹配规则。

工作区修改文件，add 选择内容，commit 留下记录，push 同步远程；协作使用分支与 PR，同步前先看 status。

表中的“文件名、地址、说明、分支名”为语法参数。连续操作示例见《Git 与 GitHub 命令速查》。

参考：[1]–[8][21]。

参考资料与官方文档（1/2）

课程资料用于基础概念与学习路径；Git 与 GitHub 官方文档用于命令、认证与平台功能的进一步查阅。点击“官方文档”可打开对应页面。

[A] Software Development Processes – 5 – Implementation & Deployment

第 4、7–12 页：版本控制、基本命令、远程同步与分支。

[B] github 小白使用指南

第 1–10 页：账号注册、仓库创建、SSH 认证与代码托管。

[1] Git: git-add / git-commit [官方文档](#)

暂存、提交与提交说明。

[2] Git: git-pull [官方文档](#)

fetch 后整合；本例显式使用 --ff-only。

[3] Git: git-merge [官方文档](#)

合并、冲突、中止与合并提交。

[4] Git: gitignore [官方文档](#)

忽略规则与已跟踪文件的区别。

[5] Git: git-restore [官方文档](#)

撤销暂存与恢复工作区。

[6] Git: git-revert [官方文档](#)

用新提交撤销旧提交。

[7] Pro Git: Branches in a Nutshell [官方文档](#)

分支、HEAD 与 switch。

[8] GitHub: Adding locally hosted code to GitHub [官方文档](#)

空远程仓库、remote 与首次 push。

[9] GitHub: Generating a new SSH key [官方文档](#)

Ed25519、口令和 ssh-agent。

[10] GitHub: Testing your SSH connection [官方文档](#)

验证账号、主机指纹与退出码。

[11] GitHub: SSH key fingerprints [官方文档](#)

首次连接时对照官方主机指纹。

Git 命令与 GitHub 平台功能分别以相应官方文档为准。

参考资料与官方文档（2/2）

本页索引涵盖提交身份、远程协作、账号安全与连接方式。主机指纹等需核验的信息，以对应官方页面为准。

[12] GitHub: Setting your commit email address [官方文档](#)

提交归属、邮箱隐私与 noreply。

[13] GitHub: About authentication to GitHub [官方文档](#)

Git 访问使用 SSH / HTTPS 认证。

[14] GitHub: Caching your GitHub credentials in Git [官方文档](#)

HTTPS + GCM / GitHub CLI。

[15] GitHub: Quickstart for pull requests [官方文档](#)

创建分支、提交更改、创建 PR。

[16] GitHub: Merging a pull request [官方文档](#)

选择合并方式并满足仓库规则。

[17] GitHub: Removing sensitive data from a repository [官方文档](#)

凭据先撤销/轮换；历史清理的边界。

[18] GitHub: Configuring two-factor authentication [官方文档](#)

双因素认证、备用方法。

[19] GitHub: Using SSH over the HTTPS port [官方文档](#)

ssh.github.com 的 443 端口备用连接。

[20] GitHub: Set up Git [官方文档](#)

安装、提交身份与认证。

[21] GitHub: About remote repositories [官方文档](#)

远程地址、origin 与 set-url。

[22] Git: 安装入口 [官方文档](#)

按操作系统选择官方安装方式。

Git 命令与 GitHub 平台功能分别以相应官方文档为准。